

Module configuration files

Ben Boeckel

<ben.boeckel@kitware.com>

version P2977R0, 2023-11-14

Table of Contents

[1. Abstract](#)

[2. Changes](#)

[2.1. R0 \(Initial\)](#)

[3. Contents](#)

[3.1. Interface source](#)

[3.2. Module name](#)

[3.3. Required modules](#)

[3.4. Required source sets](#)

[3.5. Visibility](#)

[3.6. Local preprocessor arguments](#)

[3.7. Working directory](#)

[4. Representation](#)

[4.1. Standalone](#)

[4.2. Cross-reference with Compile Commands Database](#)

[4.3. Share with Compile Commands Database](#)

[5. Availability](#)

[6. Versioning](#)

[7. References](#)

Document number	ISO/IEC/JTC1/SC22/WG21/P2977R0
Date	2023-11-14
Reply-to	Ben Boeckel, ben.boeckel@kitware.com
Audience	EWG (Evolution), SG15 (Tooling)

1. Abstract

Build tools need to be able to manage modules for projects that do not use the same compiler as the tool. However, compiled module formats are generally not compatible between compilers (or even compiler versions) or tools which may need to analyze modules for their own purposes (e.g., static analyzers). These tools need to be able to compile module interfaces in their own format with an understanding that corresponds to the actual build and therefore need to know what flags are relevant in order to make analogous compiled modules as the main build.

2. Changes

2.1. R0 (Initial)

Initial paper.

3. Contents

In order to create compatible module files for a build, a tool needs to know:

- the module interface source;
- the name of the module that is being generated;
- the names of modules that it requires;
- the set of sources which may provide modules which are required;
- the visibility of the module within the set of sources;
- the set of "local preprocessor arguments" used during the build when processing the source; and
- the working directory of the compilation.

Given this, a tool may traverse the dependency graph through the set of sources information for a given source in order to generate the module artifacts for whatever analysis is being performed.

3.1. Interface source

The interface source path is required so that the tool understands what constitutes the module itself.

3.2. Module name

The module name is required so that the corresponding source may be used to satisfy an `import` elsewhere that uses the owning module set.

3.3. Required modules

Modules imported by the module also need to be known in order to prepare satisfaction of the contained `import` statements.

3.4. Required source sets

Modules are described in terms of "module sets". Only modules that are members of the named module sets may be used to satisfy `import` requests within the current module.

3.5. Visibility

Modules that are part of a module set may be marked as "private" to indicate that they are not eligible for use by other module sets. For example, the contained symbols might not be accessible at runtime (using `-fvisibility=hidden` or a lack of `__declspec` decorations). However, given that module names must be unique program-wide, they are specified in order to give more useful diagnostics if they are mentioned.

3.6. Local preprocessor arguments

These arguments are required to be used to create the module in a corresponding way. Tooling may need to "translate" flags for the compiler in use (where the flags come from) for itself.

3.7. Working directory

Tooling interprets relative paths differently based on the current working directory. This field is specified so that tooling may agree with the compiler as needed.

4. Representation

There are a few potential ways in which this information may be represented within a build tree. Here, a few possible representations are presented.

4.1. Standalone

The first option is for a standalone database which contains all of the relevant information. It might be in separate files (see below) and later combined into a single database for convenience.

A benefit of this is that the content could be reused for the installation rather than just the build tree (as a compilation database doesn't make sense for installations).

4.2. Cross-reference with Compile Commands Database

Another way would be for the module database to be used in conjunction with a compilation database [[json-cdb](#)]. This would help to reduce duplication, but would require tooling to manually perform joins on the two databases to get all of the required information.

The main issue with this approach is that the most reliable way of correlating the module database with the compilation database relies on an optional value (**output**) as a single source might participate in a build graph more than once with different flags (e.g., building release and debug or static and shared variants at the same time).

It would also likely involve adopting the compilation database from the Clang team and into ISO with the additional enhancements specified for modules.

4.3. Share with Compile Commands Database

Another alternative would be to split the information into parts that could be shared with the compilation database (such as the local preprocessor flags) and the module-specific information (such as module sets and their dependencies) refer to this shared information as well.

This approach would not require that the compilation database be adopted into ISO as it would just also have pointers to the shared portion (though the rationale for the split may be awkward).

5. Availability

Generally, these module compilation databases must be created during the build itself. This is because the set of module names in a build are not necessarily known until the build is underway.

However, this is not a new problem as the compilation database can refer to the compilation of generated sources which do not exist until the build has completed some of its work.

Build systems should offer mechanisms to combine module compilation databases together into combined files in well-known locations so that consuming tools do not need to search for relevant files and have a reliable way to make sure that the information is consistent across the entire file. This would be provided by relying on standard features of build systems to update outputs when inputs change and the appropriate dependency information provided.

6. Versioning

There are two properties with integer values in the top-level JSON object of the format: **version** and **revision**. The **version** property is required and if **revision** is not provided, it can be assumed to be **0**. These indicate the version of the information available in the format itself and what features may be used.

The **version** integer is incremented when semantic information required for a correct interpretation is different than the previous version. When the **version** is incremented, the **revision** integer is reset to **0**.

The **revision** integer is incremented when the semantic information of the format is the same as the previous revision of the same version, but it may include additionally specified information or use an additionally specified format for the same information. For example, adding a **modification_time** or **input_hash** field may be helpful in some cases, but is not required to understand the dependency graph. Such an addition would cause an increment of the **revision** value.

The version specified in this document is:

Version fields for this specification

```
{
  "version": 1,
  "revision": 0
}
```

7. References

- [json-cdb] JSON Compilation Database Format Specification.
<https://clang.llvm.org/docs/JSONCompilationDatabase.html>.